

NgRx Product State Flow in Spartacus — Understanding & POC Guide

Table of Contents

1. *[Architecture Overview](#1-architecture-overview)*
2. *[Layer-by-Layer Breakdown](#2-layer-by-layer-breakdown)*
3. *[Full Data Flow Diagram](#3-full-data-flow-diagram)*
4. *[Key Concepts: Scoped Loading](#4-key-concepts-scoped-loading)*
5. *[POC — Integrating a Public API (FakeStore) with the Same NgRx Pattern](#5-poc--integrating-a-public-api-fakestore-with-the-same-ngrx-pattern)*

1. *Architecture Overview*

Spartacus uses a strict layered NgRx architecture for all data fetching. Every feature (product, cart, user, etc.) follows the same pattern:

~~~

*Component / Facade*

*↓ dispatch action / select state*

*NgRx Store*

*↓ action triggers effect*

*Effect → Connector → Adapter → HTTP (OCC or custom API)*

*↓ dispatches success/fail action*

*Reducer → updates state slice*

*↓*

*Selector → Component reads updated state*

~~~

*The key files for the ****product**** feature live in:*

~~~

*projects/core/src/product/*

*|—— store/*

*| |—— actions/product.action.ts ← Action classes*

*| |—— effects/product.effect.ts ← Side-effect (HTTP call)*

*| |—— reducers/index.ts ← Reducer wiring*

```

|   |—— selectors/product.selectors.ts ← Memoized selectors
|   |—— product-state.ts ← State shape interfaces
|—— connectors/product/
|   |—— product.adapter.ts ← Abstract adapter contract
|   |—— product.connector.ts ← Bridges effect ↔ adapter
|—— facade/product.service.ts ← Public API for components
|—— services/product-loading.service.ts ← Smart load-once logic

```

```

---

## ## 2. Layer-by-Layer Breakdown

### ### 2.1 State Shape (product-state.ts)

```typescript

```

export const PRODUCT_FEATURE = 'product'; // NgRx feature key
export const PRODUCT_DETAIL_ENTITY = '[Product] Detail Entity';

```

```

export interface StateWithProduct {
  [PRODUCT_FEATURE]: ProductsState; // root slice
}

```

```

export interface ProductsState {
  details: EntityScopedLoaderState<Product>; // keyed by productCode + scope
  search: ProductsSearchState;
  reviews: ProductReviewsState;
  references: ProductReferencesState;
}

```

```

`EntityScopedLoaderState<T>` is a generic wrapper that tracks:

- `loading: boolean`
- `success: boolean`
- `error: boolean | Error`
- `value: T`

for **each product code** AND **each scope** (e.g. ``list``, ``details``, ``price``).

---

### ### 2.2 Actions (`product.action.ts`)

Three action classes extend Spartacus's scoped loader base classes:

```
``typescript
// Triggers the HTTP load
export class LoadProduct extends EntityScopedLoaderActions.EntityScopedLoadAction {
  readonly type = '[Product] Load Product Data';
  constructor(public payload: string, scope = "") { // payload = productCode
    super(PRODUCT_DETAIL_ENTITY, payload, scope);
  }
}

// Dispatched when HTTP succeeds
export class LoadProductSuccess extends EntityScopedLoaderActions.EntityScopedSuccessAction {
  readonly type = '[Product] Load Product Data Success';
  constructor(public payload: Product, scope = "") {
    super(PRODUCT_DETAIL_ENTITY, payload.code ?? "", scope);
  }
}

// Dispatched when HTTP fails
export class LoadProductFail extends EntityScopedLoaderActions.EntityScopedFailAction {
  readonly type = '[Product] Load Product Data Fail';
  constructor(productCode: string, public payload: any, scope = "") {
    super(PRODUCT_DETAIL_ENTITY, productCode, scope, payload);
  }
}
``
```

The base classes automatically attach ``meta`` (`entityId`, `scope`) so reducers and selectors can key state correctly.

---

### ### 2.3 Effect (product.effect.ts)

The effect listens for `LOAD_PRODUCT`, batches concurrent requests via `bufferDebounceTime`, then calls the connector:

```
``typescript
loadProduct$ = createEffect(
  () => ({ debounce = 0 } = {}) =>
    this.actions$.pipe(
      ofType(ProductActions.LOAD_PRODUCT),
      map((action) => ({ code: action.payload, scope: action.meta.scope })),
      bufferDebounceTime(debounce), // batch simultaneous loads
      mergeMap((products) =>
        merge(
          ...this.productConnector
            .getMany(products) // delegate to connector
            .map((load) => this.productLoadEffect(load))
        )
      ),
      withdrawOn(this.contextChange$) // cancel on language/currency change
    );
``
```

`withdrawOn` is a custom RxJS operator that cancels all in-flight requests when site context changes (language/currency switch).

---

### ### 2.4 Connector (product.connector.ts)

The connector is a thin bridge between the effect and the adapter. It does **not** know about HTTP — it just delegates:



```

```typescript
@Injectable({ providedIn: 'root' })
export class ProductConnector {
  constructor(protected adapter: ProductAdapter) {}

  get(productCode: string, scope = ""): Observable<Product> {
    return this.adapter.load(productCode, scope);
  }

  getMany(products: ScopedProductData[]): ScopedProductData[] {
    if (!this.adapter.loadMany) {
      return products.map((p) => ({ ...p, data$: this.adapter.load(p.code, p.scope) }));
    }
    return this.adapter.loadMany(products);
  }
}
```

```

This is the *\*\*extension point\*\** — you swap the adapter to change the data source.

---

### ### 2.5 Adapter (occ-product.adapter.ts)

The OCC adapter implements the abstract `ProductAdapter` and makes the actual HTTP call:

```

```typescript
@Injectable()
export class OccProductAdapter implements ProductAdapter {
  load(productCode: string, scope?: string): Observable<Product> {
    return this.http
      .get(this.getEndpoint(productCode, scope))
      .pipe(this.converter.pipeable(PRODUCT_NORMALIZER)); // normalize OCC → Product model
  }

  protected getEndpoint(code: string, scope?: string): string {

```

```

return this.occEndpoints.buildUrl('product', {
  urlParams: { productCode: code },
  scope,
});
}
}
'''

```

``PRODUCT_NORMALIZER`` is a converter token — it maps the raw OCC JSON to Spartacus's ``Product`` model.

---

### ### 2.6 Reducer (`reducers/index.ts`)

The reducer is wired using ``entityScopedLoaderReducer`` — a generic factory that handles all load/success/fail state transitions automatically:

```

'''typescript
export function getReducers(): ActionReducerMap<ProductsState, any> {
  return {
    details: entityScopedLoaderReducer<Product>(PRODUCT_DETAIL_ENTITY),
    search: fromProductsSearch.reducer,
    reviews: fromProductReviews.reducer,
    references: fromProductReferences.reducer,
  };
}
'''

```

You don't write a manual switch/case for load/success/fail — the generic reducer handles it based on the ``meta.entityId`` and ``meta.scope`` attached to each action.

---

### ### 2.7 Selectors (`product.selectors.ts`)

Selectors are factory functions (because they're parameterized by ``productCode`` and ``scope``):

```

```typescript
// Get the full loader state (loading, success, error, value)
export const getSelectedProductStateFactory = (code: string, scope = "") =>
createSelector(getProductState, (details) =>
(StateUtils.entityLoaderStateSelector(details, code) as any)[scope]
// StateUtils.initialLoaderState
);

// Get just the product value
export const getSelectedProductFactory = (code: string, scope = "") =>
createSelector(
getSelectedProductStateFactory(code, scope),
(productState) => StateUtils.loaderValueSelector(productState)
);
```

```

---

### ### 2.8 Facade (product.service.ts)

The facade is what components use. It hides all NgRx complexity:

```

```typescript
@Injectable({ providedIn: 'root' })
export class ProductService {
get(productCode: string, scopes = DEFAULT_SCOPE): Observable<Product | undefined> {
return productCode
? this.productLoading.get(productCode, [].concat(scopes))
: of(undefined);
}

isLoading(productCode: string, scope = ""): Observable<boolean> {
return this.store.pipe(select(ProductSelectors.getSelectedProductLoadingFactory(productCode, scope)));
}
}
```

```

### ### 2.9 Smart Loading Service (product-loading.service.ts)

This service ensures a product is only loaded **once** even with multiple subscribers. It uses `using()` to tie the load side-effect to the observable subscription lifecycle:

```
``typescript
protected getProductForScope(productCode: string, scope: string): Observable<Product> {
  const shouldLoad$ = this.store.pipe(
    select(ProductSelectors.getSelectedProductStateFactory(productCode, scope)),
    map((state) => !state.loading && !state.success && !state.error),
    filter((x) => x)
  );

  const productLoadLogic$ = merge(shouldLoad$, ...reloadTriggers).pipe(
    debounceTime(0),
    withLatestFrom(isLoading$),
    tap([, isLoading]) => {
      if (!isLoading) this.store.dispatch(new ProductActions.LoadProduct(productCode, scope));
    }
  );

  return using(
    () => productLoadLogic$.subscribe(), // side-effect: dispatch load action
    () => productData$ // data stream returned to consumer
  ).pipe(shareReplay({ bufferSize: 1, refCount: true }));
}
``
```

---

### ## 3. Full Data Flow Diagram

...

Component calls ProductService.get('123')

|





*ProductLoadingService.get('123', ['details'])*

- checks store: not loaded yet
- dispatches *LoadProduct('123', 'details')*

|



*NgRx Effect (product.effect.ts)*

- ofType(*LOAD\_PRODUCT*)
- buffers concurrent requests
- calls *ProductConnector.getMany(...)*

|



*ProductConnector → OccProductAdapter.load('123', 'details')*

- HTTP GET */occ/v2/{baseSite}/products/123?fields=...*

|



*HTTP Response (raw OCC JSON)*

- piped through *PRODUCT\_NORMALIZER* converter

|



*Effect dispatches LoadProductSuccess({ code: '123', ...data }, 'details')*

|



*Reducer (entityScopedLoaderReducer)*

- updates *state.product.details['123']['details']*
- sets *loading=false, success=true, value=Product*

|



*Selector (getSelectedProductFactory('123', 'details'))*

- emits updated *Product* value

|



*Component receives Product via Observable*

...

---

## ## 4. Key Concepts: Scoped Loading

Spartacus loads product data in **scopes** to avoid over-fetching. Each scope maps to a different set of OCC fields:

| Scope        | Data loaded                         |
|--------------|-------------------------------------|
| "" (default) | Basic product info                  |
| `list`       | Minimal data for product lists      |
| `details`    | Full PDP data                       |
| `price`      | Price only (cleared on auth change) |
| `stock`      | Stock/availability                  |

Multiple scopes are merged in `ProductLoadingService` using `deepMerge`.

---

## ## 5. POC — Integrating a Public API (FakeStore) with the Same NgRx Pattern

This POC integrates `https://fakestoreapi.com/products/:id` using the **exact same NgRx pattern** as Spartacus's product flow, reusing the existing OCC module structure.

The FakeStore API returns:

```
```json
{ "id": 1, "title": "...", "price": 13.5, "description": "...", "category": "...", "image": "..." }
```
```

We'll map this to Spartacus's `Product` model.

---

### ### POC State Diagram (Fakestore)

---

User enters Product ID + clicks Load

|



*FakestoreProductComponent*

- calls `facade.load(id)`
- subscribes via `facade.get(id)`

|



*Action: [Fakestore] Load Product*

|



*FakestoreEffects*

- calls `FakestoreProductAdapter.load(id)`
- HTTP GET `https://fakestoreapi.com/products/:id`

|



*API Response (raw JSON)*

- normalized → Spartacus Product shape

|



*Action: [Fakestore] Load Product Success*

|



*Reducer updates state:*

`fakestore.details[id] = { loading:false, success:true, error:null, value:Product }`

|



*Selectors emit new data*

|



*Component renders updated Product card*

...

---

**### Step 1 — State Shape**

**\*\*File: `src/app/fakestore/store/fakestore-state.ts`\*\***

```

```typescript
import { Product } from '@spartacus/core';

export const FAKESTORE_FEATURE = 'fakestore';
export const FAKESTORE_DETAIL_ENTITY = '[Fakestore] Detail Entity';

export interface StateWithFakestore {
  [FAKESTORE_FEATURE]: FakestoreState;
}

export interface FakestoreState {
  details: { [id: string]: FakestoreLoaderState };
}

export interface FakestoreLoaderState {
  loading: boolean;
  success: boolean;
  error: boolean;
  value: Product | null;
}
```

```

---

### ### Step 2 — Actions

**\*\*File: `src/app/fakestore/store/actions/fakestore.actions.ts`\*\***

```

```typescript
import { createAction, props } from '@ngrx/store';
import { Product } from '@spartacus/core';

export const loadFakestoreProduct = createAction(
  '[Fakestore] Load Product',
  props<{ id: string }>()
);

```



```
export const loadFakestoreProductSuccess = createAction(
  '[Fakestore] Load Product Success',
  props<{ id: string; product: Product }>()
);
```

```
export const loadFakestoreProductFail = createAction(
  '[Fakestore] Load Product Fail',
  props<{ id: string; error: any }>()
);
```

```
---
```

```
---
```

### ### Step 3 — Adapter (HTTP layer)

```
**File: `src/app/fakestore/adapters/fakestore-product.adapter.ts`**
```

```
```typescript
```

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import { Product } from '@spartacus/core';
```

```
@Injectable({ providedIn: 'root' })
export class FakestoreProductAdapter {
  private readonly baseUrl = 'https://fakestoreapi.com/products';
```

```
  constructor(private http: HttpClient) {}
```

```
  load(id: string): Observable<Product> {
    return this.http.get<any>(`${this.baseUrl}/${id}`).pipe(
      map((item) => this.normalize(item))
    );
  }
}
```

```

private normalize(item: any): Product {
  return {
    code: String(item.id),
    name: item.title,
    description: item.description,
    price: { value: item.price, currencyIso: 'USD', formattedValue: `$$${item.price}` },
    categories: [{ name: item.category }],
    images: [{ url: item.image, format: 'product' }],
  };
}
}
}

```

---

### ### Step 4 — Effect

**\*\*File: `src/app/fakestore/store/effects/fakestore.effects.ts`\*\***

```

``typescript
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { of } from 'rxjs';
import { catchError, map, mergeMap } from 'rxjs/operators';
import { FakestoreProductAdapter } from '../adapters/fakestore-product.adapter';
import * as FakestoreActions from '../actions/fakestore.actions';

@Injectable()
export class FakestoreEffects {
  loadProduct$ = createEffect(() =>
    this.actions$.pipe(
      ofType(FakestoreActions.loadFakestoreProduct),
      mergeMap(({ id }) =>
        this.adapter.load(id).pipe(
          map((product) => FakestoreActions.loadFakestoreProductSuccess({ id, product })),
          catchError((error) => of(FakestoreActions.loadFakestoreProductFail({ id, error })))
        )
      )
    )
  );
}

```

```
)  
)  
)  
);
```

```
constructor(  
  private actions$: Actions,  
  private adapter: FakestoreProductAdapter  
) {}  
}  
...
```

```
---
```

### ### Step 5 — Reducer

```
**File: `src/app/fakestore/store/reducers/fakestore.reducer.ts`**
```

```
``typescript
```

```
import { createReducer, on } from '@ngrx/store';  
import { FakestoreState } from '../fakestore-state';  
import * as FakestoreActions from '../actions/fakestore.actions';
```

```
const initialState: FakestoreState = { details: {} };
```

```
export const fakestoreReducer = createReducer(  
  initialState,  
  on(FakestoreActions.loadFakestoreProduct, (state, { id }) => ({  
    ...state,  
    details: { ...state.details, [id]: { loading: true, success: false, error: false, value: null } },  
  })),  
  on(FakestoreActions.loadFakestoreProductSuccess, (state, { id, product }) => ({  
    ...state,  
    details: { ...state.details, [id]: { loading: false, success: true, error: false, value: product } },  
  })),  
  on(FakestoreActions.loadFakestoreProductFail, (state, { id }) => ({
```

```

...state,
details: { ...state.details, [id]: { loading: false, success: false, error: true, value: null } },
}))
);
'''

```

---

### ### Step 6 — Selectors

**\*\*File: `src/app/fakestore/store/selectors/fakestore.selectors.ts`\*\***

```

'''typescript
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { FAKESTORE_FEATURE, FakestoreState } from '../fakestore-state';

const getFakestoreState = createFeatureSelector<FakestoreState>(FAKESTORE_FEATURE);

export const getFakestoreProductState = (id: string) =>
  createSelector(getFakestoreState, (state) => state.details[id]);

export const getFakestoreProduct = (id: string) =>
  createSelector(getFakestoreProductState(id), (s) => s?.value ?? null);

export const isFakestoreProductLoading = (id: string) =>
  createSelector(getFakestoreProductState(id), (s) => s?.loading ?? false);
'''

```

---

### ### Step 7 — Facade Service

**\*\*File: `src/app/fakestore/facade/fakestore-product.service.ts`\*\***

```

'''typescript
import { Injectable } from '@angular/core';

```

```
import { select, Store } from '@ngrx/store';
import { Observable } from 'rxjs';
import { tap, filter } from 'rxjs/operators';
import { Product } from '@spartacus/core';
import { StateWithFakestore } from '../store/fakestore-state';
import * as FakestoreActions from '../store/actions/fakestore.actions';
import * as FakestoreSelectors from '../store/selectors/fakestore.selectors';
```

```
@Injectable({ providedIn: 'root' })
export class FakestoreProductService {
  constructor(private store: Store<StateWithFakestore>) {}

  get(id: string): Observable<Product | null> {
    return this.store.pipe(
      select(FakestoreSelectors.getFakestoreProductState(id)),
      tap((state) => {
        if (!state || (!state.loading && !state.success && !state.error)) {
          this.store.dispatch(FakestoreActions.loadFakestoreProduct({ id }));
        }
      }),
      filter((state) => !!state?.success),
      select(FakestoreSelectors.getFakestoreProduct(id))
    );
  }
}
```

```
isLoading(id: string): Observable<boolean> {
  return this.store.pipe(select(FakestoreSelectors.isFakestoreProductLoading(id)));
}

}

""
```

---

### ### Step 8 — NgRx Module Registration

```
**File: `src/app/fakestore/fakestore-store.module.ts`**
```



```

```typescript
import { NgModule } from '@angular/core';
import { StoreModule } from '@ngrx/store';
import { EffectsModule } from '@ngrx/effects';
import { FAKESTORE_FEATURE } from './store/fakestore-state';
import { fakestoreReducer } from './store/reducers/fakestore.reducer';
import { FakestoreEffects } from './store/effects/fakestore.effects';

```

```

@NgModule({
  imports: [
    StoreModule.forFeature(FAKESTORE_FEATURE, fakestoreReducer),
    EffectsModule.forFeature([FakestoreEffects]),
  ],
})
export class FakestoreStoreModule {}
```

```

Import `FakestoreStoreModule` in your `AppModule` or a lazy-loaded feature module.

---

### ### Step 9 — Component Usage

**\*\*File: `src/app/fakestore/components/fakestore-product.component.ts`\*\***

```

```typescript
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { Product } from '@spartacus/core';
import { FakestoreProductService } from '../facade/fakestore-product.service';

@Component({
  selector: 'app-fakestore-product',
  template: `
    <ng-container *ngIf="isLoading$ | async">Loading...</ng-container>
    <ng-container *ngIf="product$ | async as product">

```

```

<h2>{{ product.name }}</h2>
<p>{{ product.description }}</p>
<strong>{{ product.price?.formattedValue }}</strong>
</ng-container>
`
;
})

export class FakestoreProductComponent implements OnInit {
  product$: Observable<Product | null>;
  isLoading$: Observable<boolean>;

  constructor(private fakestoreService: FakestoreProductService) {}

  ngOnInit(): void {
    this.product$ = this.fakestoreService.get('');
    this.isLoading$ = this.fakestoreService.isLoading('');
  }
}

```

## ## Summary: Spartacus NgRx Pattern Checklist

| Layer          | Spartacus file            | Your custom file               |
|----------------|---------------------------|--------------------------------|
| State shape    | `product-state.ts`        | `fakestore-state.ts`           |
| Actions        | `product.action.ts`       | `fakestore.actions.ts`         |
| Effect         | `product.effect.ts`       | `fakestore.effects.ts`         |
| Adapter (HTTP) | `occ-product.adapter.ts`  | `fakestore-product.adapter.ts` |
| Reducer        | `reducers/index.ts`       | `fakestore.reducer.ts`         |
| Selectors      | `product.selectors.ts`    | `fakestore.selectors.ts`       |
| Facade         | `product.service.ts`      | `fakestore-product.service.ts` |
| Module         | `product-store.module.ts` | `fakestore-store.module.ts`    |

The key difference from Spartacus's built-in product flow is that Spartacus uses the abstract `ProductAdapter` + `ProductConnector`` indirection to allow swapping the HTTP implementation. For a custom API you can simplify

*by injecting the adapter directly in the effect, as shown in this POC.*